TECHNICAL CURRICULUM & ARCHITECTURES

NLP, Transformers, LLMs & Prompt Engineering

A Comprehensive Technical Guide to Language Architecture & Generative Models

1. Foundations of Natural Language Processing & Text Representation

Natural Language Processing (NLP) bridges raw unstructured text data and structured numerical vector representations. Modern language systems map discrete linguistic tokens into high-dimensional continuous semantic vector spaces $X \in \mathbb{R}^{N \times d}$.

This technical reference details the mathematical principles, neural transformer architectures, large language model scaling paradigms, parameter-efficient fine-tuning techniques, and structured prompt engineering methodologies that power current state-of-the-art NLP engines.

> **The Fundamental Paradigm of Language Modeling**
> Autoregressive language modeling formulates text generation as estimating joint probability distributions over token sequences using the chain rule of probability: $P(w_1, w_2, \ldots, w_N) = \prod_{i=1}^{N} P(w_i \mid w_1, \ldots, w_{i-1})$.

2. Tokenization & Vector Embeddings

Tokenization converts unstructured text into discrete integer token IDs. Modern LLMs use subword tokenization algorithms to handle open vocabularies and out-of-vocabulary (OOV) terms cleanly.

2.1 Subword Tokenization Algorithms

Algorithm	Splitting Strategy	Vocabulary Construction	Primary Architecture Use
Byte-Pair Encoding (BPE)	Iterative merging of frequent byte/character pairs	Bottom-up frequency-based merge table	GPT-2 / GPT-4, RoBERTa, LLaMA
WordPiece	Merges characters maximizing likelihood of training data	Likelihood gain ratio scoring	BERT, DistilBERT
Unigram Language Model	Starts with large vocabulary and prunes low-probability tokens	Probabilistic unigram loss reduction	SentencePiece, T5, ALBERT

2.2 Vectorized Subword Tokenizer Simulation

Below is an explicit Python implementation illustrating subword character-pair merging logic central to BPE tokenizers.

```
import re
from collections import Counter, defaultdict

def get_stats(vocab: dict[tuple[str, ...], int]) -> dict[tuple[str, str], int]:
    """Count frequency of adjacent symbol pairs in vocabulary dictionary."""
    pairs = defaultdict(int)
    for word, freq in vocab.items():
        for i in range(len(word) - 1):
            pairs[word[i], word[i+1]] += freq
    return pairs

def merge_vocab(pair: tuple[str, str], v_in: dict[tuple[str, ...], int]) ->
dict[tuple[str, ...], int]:
    """Merge all occurrences of the target symbol pair in the vocabulary."""
    v_out = {}
    bigram = re.escape(' '.join(pair))
    pattern = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
    for word in v_in:
        w_str = ' '.join(word)
        w_merged = pattern.sub(''.join(pair), w_str)
        v_out[tuple(w_merged.split())] = v_in[word]
    return v_out

# Example BPE Execution Iteration
corpus_vocab = {('l', 'e', 'a', 'r', 'n'): 5, ('l', 'e', 'a', 'r', 'n', 'e', 'r'): 2}
pair_counts = get_stats(corpus_vocab)
best_pair = max(pair_counts, key=pair_counts.get)
updated_vocab = merge_vocab(best_pair, corpus_vocab)
print("Best Symbol Pair Merged:", best_pair)
```

3. Sequence Modeling: From RNNs to Attention Mechanisms

Prior to Transformers, sequential architectures processed text sequentially, accumulating hidden state vectors over time.

3.1 Recurrent Neural Networks & Gated Architectures

Standard Recurrent Neural Networks (RNNs) suffer from vanishing and exploding gradients when processing long contexts. Long Short-Term Memory (LSTM) units introduce gated memory cells to regulate information flow:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad | \quad i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad | \quad C_t = f_t \circledcirc C_{t-1} + i_t \circledcirc \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

3.2 Scaled Dot-Product Self-Attention

Attention removes sequential bottlenecks, enabling parallel computation across arbitrary sequence lengths. Queries (Q), Keys (K), and Values (V) interact via matrix projection:

$$Attention(Q, K, V) = Softmax((Q K^T) / \sqrt{d_k}) V$$

```python
import numpy as np

def scaled_dot_product_attention(Q: np.ndarray, K: np.ndarray, V: np.ndarray, mask: np.ndarray = None) -> tuple:
    """Compute Scaled Dot-Product Self-Attention using pure vectorized NumPy."""
    d_k = Q.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)

    if mask is not None:
        scores = np.where(mask == 0, -1e9, scores)

    weights = np.exp(scores - np.max(scores, axis=-1, keepdims=True))
    weights /= np.sum(weights, axis=-1, keepdims=True)

    output = weights @ V
    return output, weights
```

> **Why Scale by √d_k?**
> For large query/key projection dimensions d_k, dot products grow large in magnitude, pushing the Softmax function into regions with tiny gradients. Scaling by $1/\sqrt{d_k}$ stabilizes variance to 1.0.

4. The Transformer Architecture

Introduced in Vaswani et al. (2017), the Transformer replaces recurrent layers with Multi-Head Attention (MHA) and Position-Wise Feed-Forward Networks (FFN).

4.1 Multi-Head Attention (MHA) & Positional Encoding

Multi-Head Attention projects input representations into multiple sub-spaces, letting the model attend to information from different representation spaces simultaneously:

$$MHA(Q, K, V) = Concat(head_1, ..., head_h)\, W^O \text{ where } head_i = Attention(Q\, W_i^Q, K\, W_i^K, V\, W_i^V)$$

Positional Encoding Scheme	Mathematical Formulation	Properties & Use Cases
Sinusoidal (Absolute)	$PE_{(pos,\ 2i)} = sin(pos\ /\ 10000^{2i/d})$	Fixed non-learnable functions; extrapolates to arbitrary sequence lengths.
Rotary Embeddings (RoPE)	Rotates Query & Key vectors in 2D planes by relative distance	Used in modern LLMs (LLaMA, Mistral); preserves relative distance decay.
ALiBi	Adds static linear penalty bias directly to attention matrix: $-m \cdot \lvert i - j \rvert$	Extrapolates efficiently to sequence lengths unseen during pretraining.

```
class MultiHeadAttentionNumPy:
    """Vectorized Multi-Head Self-Attention Layer Implementation."""
    def __init__(self, embed_dim: int, num_heads: int):
        self.embed_dim = embed_dim
        self.num_heads = num_heads
        self.head_dim = embed_dim // num_heads

        self.W_q = np.random.randn(embed_dim, embed_dim) * 0.02
        self.W_k = np.random.randn(embed_dim, embed_dim) * 0.02
        self.W_v = np.random.randn(embed_dim, embed_dim) * 0.02
        self.W_o = np.random.randn(embed_dim, embed_dim) * 0.02

    def forward(self, x: np.ndarray, mask: np.ndarray = None) -> np.ndarray:
        batch_seq, _ = x.shape
        Q = x @ self.W_q
        K = x @ self.W_k
        V = x @ self.W_v

        # Reshape to (num_heads, seq_len, head_dim)
        Q_h = Q.reshape(batch_seq, self.num_heads, self.head_dim).swapaxes(0, 1)
        K_h = K.reshape(batch_seq, self.num_heads, self.head_dim).swapaxes(0, 1)
        V_h = V.reshape(batch_seq, self.num_heads, self.head_dim).swapaxes(0, 1)

        # Attention computation per head
        out_heads = []
        for h in range(self.num_heads):
            head_out, _ = scaled_dot_product_attention(Q_h[h], K_h[h], V_h[h], mask)
            out_heads.append(head_out)

        concat_out = np.concatenate(out_heads, axis=-1)
        return concat_out @ self.W_o
```

5. Encoder-Only, Decoder-Only, and Encoder-Decoder Paradigms

Transformer components can be arranged into distinct architectural families depending on the downstream target objective.

5.1 Structural Taxonomy

Architecture Type	Attention Masking	Pretraining Objective	Representative Models
Encoder-Only	Bidirectional (Full Context)	Masked Language Modeling (MLM)	BERT, RoBERTa, DeBERTa
Decoder-Only	Causal (Lower Triangular Mask)	Next-Token Prediction (Autoregressive)	GPT-3/4, LLaMA, Mistral, Claude
Encoder-Decoder	Bidirectional Encoder + Causal Decoder	Sequence-to-Sequence Span Masking	T5, BART, Whisper

5.2 Causal Masking Mechanism

Autoregressive LLMs prevent tokens from attending to future tokens during training by applying an upper-triangular mask of negative infinity values prior to the Softmax operation:

```
def create_causal_mask(seq_len: int) -> np.ndarray:
    """Generate lower-triangular binary causal mask for autoregressive generation."""
    mask = np.tril(np.ones((seq_len, seq_len)))
    return mask

# Visualizing Causal Mask Matrix (1 = Keep, 0 = Mask out)
mask_matrix = create_causal_mask(4)
print("4x4 Causal Mask Matrix:\n", mask_matrix)
```

> **The Shift to Decoder-Only Architectural Dominance**
>
> Decoder-only autoregressive models scale more predictably via empirical Scaling Laws ($L(N, D) = (N_c/N)^{\alpha_N} + (D_c/D)^{\alpha_D}$), outperforming encoder-decoder setups on zero-shot generalization across diverse tasks.

6. Large Language Models (LLMs) & Efficiency Innovations

Scaling LLMs to hundreds of billions of parameters introduces memory bandwidth bottlenecks, prompting key architectural innovations in attention compute efficiency and KV-caching.

6.1 KV-Cache & Efficient Attention Variants

During inference generation, previous key-value projections are cached to avoid redundant $O(N^2)$ recomputations at each new step.

Attention Variant	Key-Value Head Structure	Memory Footprint	Throughput Gain
Multi-Head Attention (MHA)	$N_{KV} = N_Q$ (Dedicated KV head per Q head)	100% Baseline	Standard Baseline
Grouped-Query Attention (GQA)	$N_{KV} = N_Q / G$ (Shared KV head per group)	Substantially Reduced (~12-25%)	High (~3-5x decoding speedup)
Multi-Query Attention (MQA)	$N_{KV} = 1$ (Single KV head shared across all Q heads)	Minimal (~8x memory drop)	Maximal Memory Efficiency

```
def kv_cache_inference_step(x_new: np.ndarray, kv_cache: dict, W_k: np.ndarray, W_v:
np.ndarray) -> tuple:
    """Appends new token projections to persistent Key-Value cache tensor."""
    k_new = x_new @ W_k
    v_new = x_new @ W_v

    if "K" in kv_cache:
        kv_cache["K"] = np.concatenate([kv_cache["K"], k_new], axis=0)
        kv_cache["V"] = np.concatenate([kv_cache["V"], v_new], axis=0)
    else:
        kv_cache["K"] = k_new
        kv_cache["V"] = v_new

    return kv_cache["K"], kv_cache["V"]
```

7. Parameter-Efficient Fine-Tuning (PEFT) & Alignment

Full parameter fine-tuning of massive models is computationally prohibitive. PEFT methods freeze base weights and update low-rank adapters, while alignment techniques align raw text predictors with human intent.

7.1 Low-Rank Adaptation (LoRA)

LoRA decomposes weight update matrices $\Delta W \in \mathbb{R}^{d \times k}$ into two lower-rank matrices $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ where rank $r \ll min(d, k)$:

$$h = W_0 x + \Delta W x = W_0 x + (\alpha / r) B A x$$

```
class LoRALayerNumPy:
    """Low-Rank Adaptation (LoRA) wrapper around a frozen linear layer."""
    def __init__(self, in_features: int, out_features: int, rank: int = 4, alpha: float =
16.0):
        self.W0 = np.random.randn(out_features, in_features) * 0.02  # Frozen Base Weights
        self.A = np.random.randn(rank, in_features) * 0.01          # Learnable Rank A
        self.B = np.zeros((out_features, rank))                     # Learnable Rank B
        self.scaling = alpha / rank

    def forward(self, x: np.ndarray) -> np.ndarray:
        base_out = x @ self.W0.T
        lora_out = (x @ self.A.T @ self.B.T) * self.scaling
        return base_out + lora_out
```

7.2 Model Alignment: RLHF & Direct Preference Optimization (DPO)

Alignment Technique	Loss Objective Formulation	Operational Complexity
RLHF (PPO)	$max_\theta E [(R(x, y) - \beta D_{KL}(\pi_\theta \| \pi_{ref}))]$	High: Requires separate Reward Model, Value Model, and Reference Model.
Direct Preference Optimization (DPO)	$L_{DPO} = -E [log \sigma(\beta log(\pi_\theta(y_w\|x)/ \pi_{ref}(y_w\|x)) - \beta log(\pi_\theta(y_l\|x)/\pi_{ref}(y_l\|x)))]$	Low: Direct loss optimization on pairwise preferences without a separate reward model.

8. Structured Prompt Engineering & Reasoning Frameworks

Prompt engineering systematically structures input contexts to steer LLM reasoning, activate specialized in-context circuits, and enforce deterministic output schemas.

8.1 Advanced Prompting Paradigms

- **Few-Shot In-Context Learning:** Provides input-output examples directly within the prompt context without gradient updates.
- **Chain-of-Thought (CoT):** Elicits step-by-step reasoning sequences prior to producing final answers ($Prompt \rightarrow Reasoning\ Steps \rightarrow Output$).
- **Tree-of-Thoughts (ToT):** Explores multiple reasoning paths systematically using search algorithms (BFS/DFS) and self-evaluation evaluation prompts.
- **ReAct (Reasoning + Acting):** Interleaves thought traces with external tool API executions to ground model outputs in real-time information.

8.2 Structured Prompt Construction Framework

```
### SYSTEM PROMPT
Role: Expert Financial Risk Analyst LLM Engine
Task: Evaluate quarterly earnings report metrics and output JSON structured assessments.
Constraints:
1. Use ONLY metrics present in the provided context.
2. Return strictly valid JSON formatted against the target schema.
3. Show step-by-step reasoning in a separate "reasoning" key before outputting verdict.

### FEW-SHOT EXAMPLES
User Input: "Revenue grew 12% to $4.2B, but net margin fell from 18% to 11% due to rising operational expenses."
Response: {
  "reasoning": "Revenue growth is positive (+12%), but margin compression (-7%) signals efficiency risk.",
  "risk_score": 0.65,
  "verdict": "MODERATE_RISK"
}

### TARGET INPUT
User Input: "Q3 revenue dropped 5% YoY to $1.1B. Debt-to-equity increased to 2.4x."
Response:
```

9. Synthesis: Complete Autoregressive Transformer Engine

Below is an end-to-end Python engine integrating embedding layers, multi-head attention, causal masking, layer normalization, residual connections, and token sampling into a complete execution pipeline.

```
class MiniAutoregressiveLLM:
    """Complete Single-Block Autoregressive Language Model Pipeline in NumPy."""
    def __init__(self, vocab_size: int = 100, embed_dim: int = 32, num_heads: int = 4):
        self.vocab_size = vocab_size
        self.token_embeddings = np.random.randn(vocab_size, embed_dim) * 0.02
        self.pos_embeddings = np.random.randn(128, embed_dim) * 0.02
        self.attention = MultiHeadAttentionNumPy(embed_dim, num_heads)
        self.lm_head = np.random.randn(embed_dim, vocab_size) * 0.02

    def layer_norm(self, x: np.ndarray, eps: float = 1e-5) -> np.ndarray:
        mean = np.mean(x, axis=-1, keepdims=True)
        var = np.var(x, axis=-1, keepdims=True)
        return (x - mean) / np.sqrt(var + eps)

    def forward(self, input_ids: np.ndarray) -> np.ndarray:
        seq_len = len(input_ids)
        x = self.token_embeddings[input_ids] + self.pos_embeddings[:seq_len]

        # Causal Attention Block with Residual & LayerNorm
        norm_x = self.layer_norm(x)
        causal_mask = create_causal_mask(seq_len)
        attn_out = self.attention.forward(norm_x, mask=causal_mask)
        x = x + attn_out

        # Projection to Vocabulary Logits
        logits = self.layer_norm(x) @ self.lm_head
        return logits

    def generate(self, prompt_ids: list[int], max_new_tokens: int = 5, temperature: float
= 0.7) -> list[int]:
        generated = list(prompt_ids)
        for _ in range(max_new_tokens):
            logits = self.forward(np.array(generated))
            next_token_logits = logits[-1, :] / temperature
            probs = np.exp(next_token_logits) / np.sum(np.exp(next_token_logits))
            next_token = int(np.random.choice(len(probs), p=probs))
            generated.append(next_token)
        return generated

# Model Verification Execution
if __name__ == "__main__":
    llm = MiniAutoregressiveLLM(vocab_size=50, embed_dim=16, num_heads=2)
    prompt = [1, 5, 12]
    output_tokens = llm.generate(prompt, max_new_tokens=4)
    print("Prompt Token Sequence:", prompt)
    print("Generated Token Sequence:", output_tokens)
```

10. Summary & NLP Architectural Roadmap

Core Language Intelligence Technical Competencies

- **Tokenization & Vectors:** BPE/WordPiece Subword Merges, Vector Embeddings, Cosine Distance.
- **Attention & Transformers:** Scaled Dot-Product Attention, Multi-Head Attention, Causal Masks, RoPE.
- **LLM Efficiency:** KV-Caching, Grouped-Query Attention (GQA), FlashAttention Mechanics.
- **Fine-Tuning & Alignment:** Low-Rank Adaptation (LoRA), RLHF (PPO), Direct Preference Optimization (DPO).
- **Prompt Engineering:** System Prompting, Few-Shot Formatting, Chain-of-Thought (CoT), ReAct Tool Execution.